

BMCS2063 Data Structures and Algorithms

Applications of Abstract Data Types (ADTs)

Chapter 1

Learning Outcomes

At the end of this lecture, you should be able to

- Explain the benefits of *abstraction*.
- Apply the use of the *list*, *stack* and *queue*.
ADTs appropriately to solve problems.

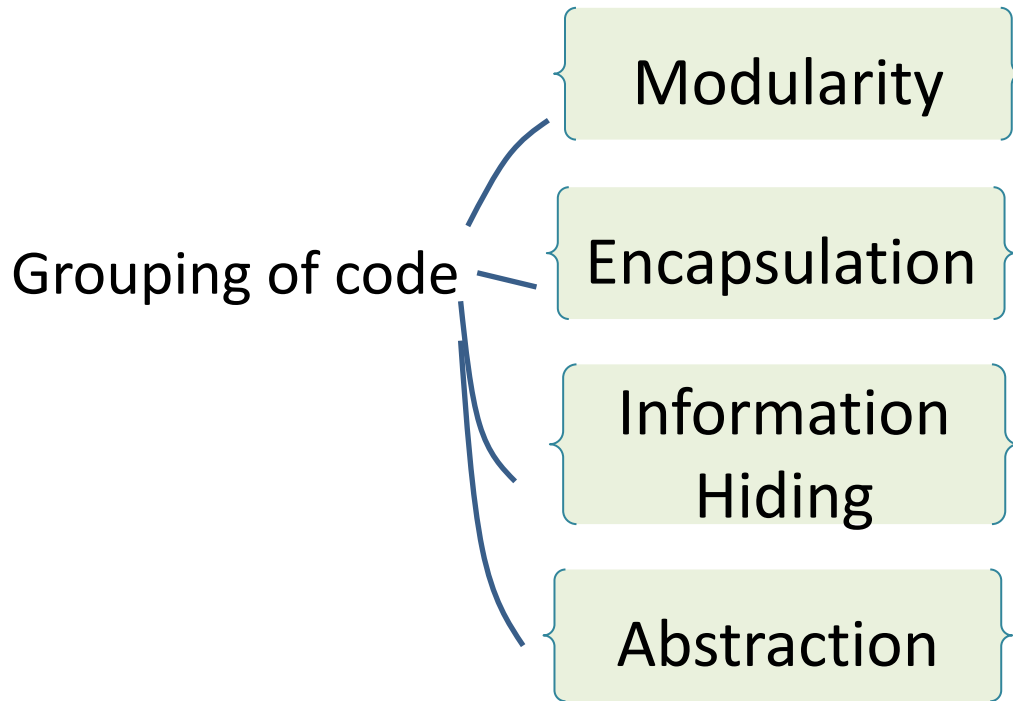
Recap of Programming Paradigms

Procedural paradigm

Object-oriented paradigm

- Which came first?
- What is the main difference?
- Why was the newer paradigm introduced?

Terminology Recap



- What do these terms mean?
- Are they present in both procedural and OO paradigms?
- Why are they important?

What is Abstraction?

ABSTRACTION

Abstract:

Exist in
thought, but
no concrete
existence

... Oxford Dictionaries

INTEGER

1

39

65

15

Why Abstraction?

ABSTRACTION

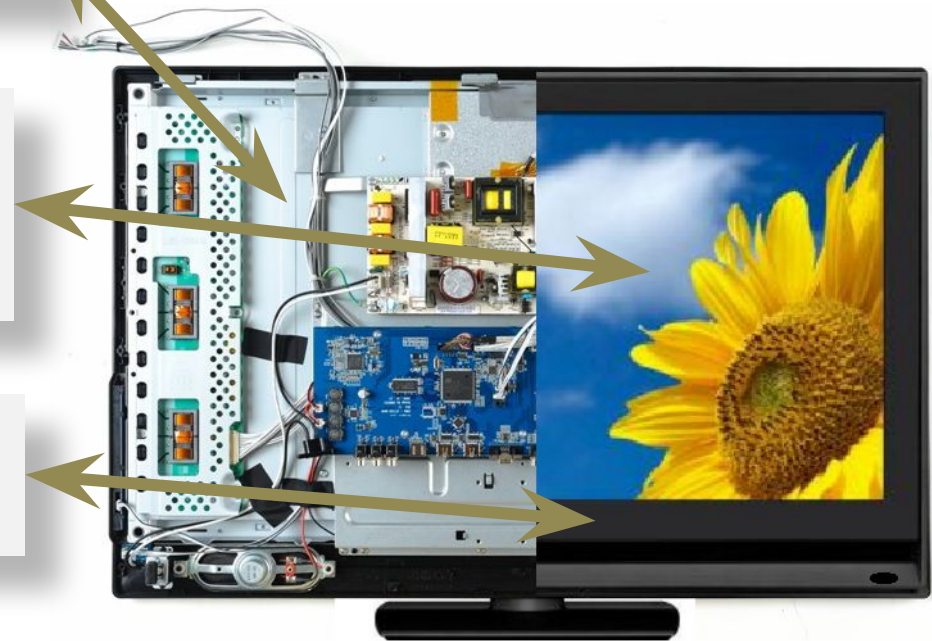
Hiding the implementation details, by providing 1 layer over the basic functionality

- ✓ User can focus on the “big picture”, not the details.
- ✓ Developer can modify the class implementation without affecting the user.

Implementation

Abstraction
(Interface and Operation)

Encapsulation
(hiding details)



Example:
`Integer.parseInt()`

Design Principles: Encapsulation

- Different components of a software system should not reveal the internal details of their respective implementations (information hiding)
- Advantage
 - Programmer has freedom in implementing the details of a system
 - Programmer only needs to maintain the contract (interface) that outsiders see

Design Principles: **Modularity**

- An organizational principle for code in which different components of a software system are divided into separate functional units.
- Rationale
 - Modern software systems consist of several different components that must interact correctly in order for the entire system to work properly.
 - Keeping these interactions straight requires that these different components be well-organized

Design Principles: **Abstraction**

- To distill a complicated system down to its most fundamental parts and
- describe these parts in a simple, precise language by naming the parts and explaining their functionality.

Abstract Data Type (ADT)



**Abstract Data
Type (ADT)**

Abstract Data Type:

A TYPE that is conceived
apart from concrete
reality

An ADT consists of data that has
certain characteristics + a set of operations that can
be used to manipulate the data

Examples of ADTs

- Integer
- Double
- String
- Fraction
- Counter

Data Structure

- The implementation of ADT is often referred to as data structure, using some constructs and primitive data types.
- A representation of data and the operations allowed on that data (Weiss, 2010)

Collection ADTs

- A general term for an ADT that contains a group of objects.
 - Some collections allow duplicate items, some do not.
 - Some collections arrange their contents in a certain order, while others do not.

How are these collections different?



Sides & Drinks	
Cookies	\$0.65
Brownie	\$1.30
Chips	\$1.30
Apple	\$1.30
Danon Light & Fit	
Strawberry Yogurt..	\$1.30
Vitamin Water Zero..	\$2.50
Powerade	\$2.00
20 oz Dasani Water ..	\$1.60
20 oz Soda Bottle.....	\$1.70
Red Bull	\$3.50
Bottled Juice	
Minute Maid	\$2.00

(B)

(C)



Algorithm

- A list of steps to solve a problem (perform as task).
- Different algorithms may be used to solve the same problem.



vs.



Resources?



Time?



The algorithm determines

Time Efficiency

- **Time** taken to complete a task

Space Efficiency

- The use of resources

Recall: Benefits of ADTs

- Reusability
- Maintainability

Introduction to Collections

- Recall: a **collection** is a general term for an ADT that contains a group of objects.
- A linear collection is a collection that stores its entries in a linear sequence.
- Examples of linear collections: List, Stack, Queue, etc.
 - They differ in the restrictions they place on how these entries may be added, removed, or accessed

Lists

List: Definition

- A list is a linear data structure used to store an ordered collection of elements
- A linear collection of entries in which entries may be added, removed, and searched for without restriction
- a data structure that represents an ordered collection

List: Basic Operations

add(<i>e</i>)	:	Insert element <i>e</i> , at the end of the list
remove(<i>i</i>)	:	Remove the element at position <i>i</i> from the list and return this element
isEmpty()	:	Return <i>true</i> if the list is empty
get(<i>i</i>)	:	Return the element at position <i>i</i> in the list, without removing it
clear()	:	Remove all the elements from the list so that the list is now empty

List: Applications

- Lists of things – task lists, list of names, playlists, etc
- High-Precision Arithmetics – to represent very long integer values

Note

- To illustrate the use of collection ADTs, we will use predefined Java interfaces and classes from the Java Collections Framework.
- In Chapter 4 onwards, you will learn how to create your own collection ADTs.

Java Collections Framework:

java.util.List interface (1)

Return Type	Method and Description
boolean	add(E e) Appends the specified element to the end of this list.
void	add(int index, E element) Inserts the specified element at the specified position in this list.
void	clear() Removes all of the elements from this list.
boolean	contains(Object o) Returns true if this list contains the specified element.

Java Collections Framework:

java.util.List interface (2)

Return Type	Method and Description
E	get (int index) Returns the element at the specified position in this list.
boolean	<u>isEmpty</u> () Returns true if this list contains no elements.
E	remove (int index) Removes the element at the specified position in this list.
int	size () Returns the number of elements in this list.

«interface»

java.util.List<E>

+*add(index: int, element: E): boolean*
+*addAll(index: int, c: Collection<? extends E>): boolean*
+*get(index: int): E*
+*indexOf(element: Object): int*
+*lastIndexOf(element: Object): int*
+*listIterator(): ListIterator<E>*
+*listIterator(startIndex: int): ListIterator<E>*
+*remove(index: int): E*
+*set(index: int, element: E): E*
+*subList(fromIndex: int, toIndex: int): List<E>*

Adds a new element at the specified index.

Adds all the elements in c to this list at the specified index.

Returns the element in this list at the specified index.

Returns the index of the first matching element.

Returns the index of the last matching element.

Returns the list iterator for the elements in this list.

Returns the iterator for the elements from startIndex to endIndex.

Removes the element at the specified index.

Sets the element at the specified index.

Returns a sublist from fromIndex to toIndex.



java.util.ArrayList<E>

+*ArrayList()*
+*ArrayList(c: Collection<? extends E>)*
+*ArrayList(initialCapacity: int)*
+*trimToSize(): void*

Creates an empty list with the default initial capacity.

Creates an array list from an existing collection.

Creates an empty list with the specified initial capacity.

Trims the capacity of this ArrayList instance to be the list's current size.

Sample Code:

Shopping List App

In Chapter1\examples\

- **ShoppingListApp.java**

Problem: Registration of Runners in a Marathon



"Wait a minute!"

Sample Code:

Registration of Marathon Runners

In Chapter1\runningman\

- **Runner.java**
- **Registration.java**

Note: Types of Lists

- *Unordered* lists
 - Entries are not arranged in any particular order
 - By default, “lists” refer to unordered lists
- *Sorted (ordered)* lists
 - Entries are maintained in sorted order
 - We will cover sorted lists in Chapter 7.

Stacks

Stack: Definition

- A collection of objects in which the objects are inserted and removed according to the last-in, first out (*LIFO*) principle A linear collection of entries in which entries may be only removed in the reverse order in which they are added, i.e. the last entry added is the first one to be removed
- Typically, there is not provision to search for an entry in the stack

Stack: Basic Operations

push (<i>e</i>)	:	Insert element <i>e</i> , to be the top of the stack
pop()	:	Remove and return the element at the top of the stack
isEmpty()	:	Return <i>true</i> if the stack is empty
peek()	:	Return the top element in the stack, without removing it
clear()	:	Remove all the elements from the stack so that the stack is now empty

Stack Trivia

- The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser



Stack: Application

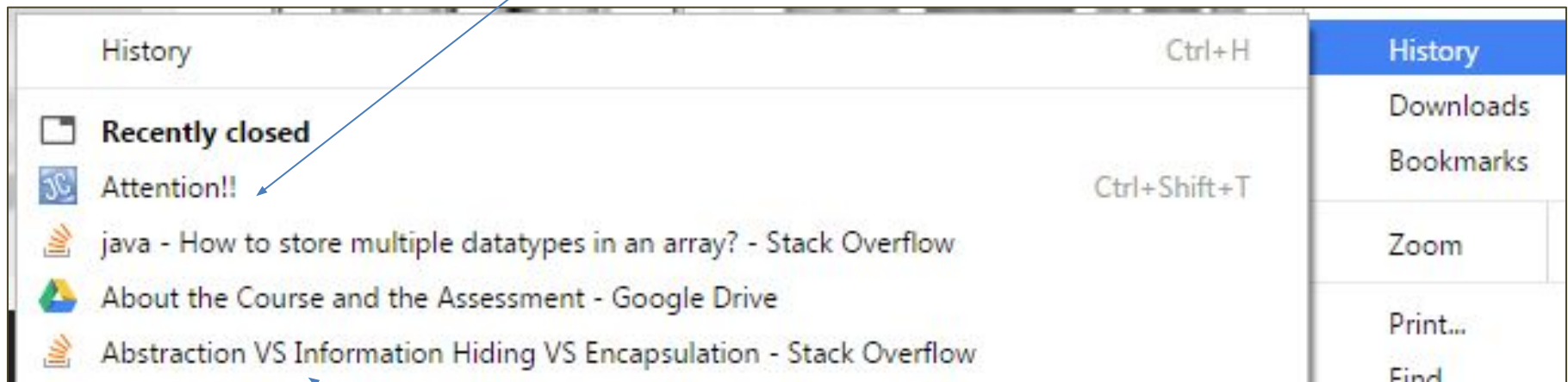


STRUCTURE**E**|

First In Last In
Last Out First Out

Stack: Application

Last In



First In

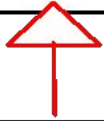
Stack: Applications

- Any application that requires LIFO processing
 - checking for palindromes (e.g. “noon”, “kayak”), reversing a string, etc.
- Program stack – to keep track of method calls

Java Collections Framework:

java.util.Stack class

java.util.Vector<E>



java.util.Stack<E>

+Stack()
+empty(): boolean
+peek(): E
+pop(): E
+push(o: E) : E
+search(o: Object) : int

Creates an empty stack.

Returns true if this stack is empty.

Returns the top element in this stack.

Returns and removes the top element in this stack.

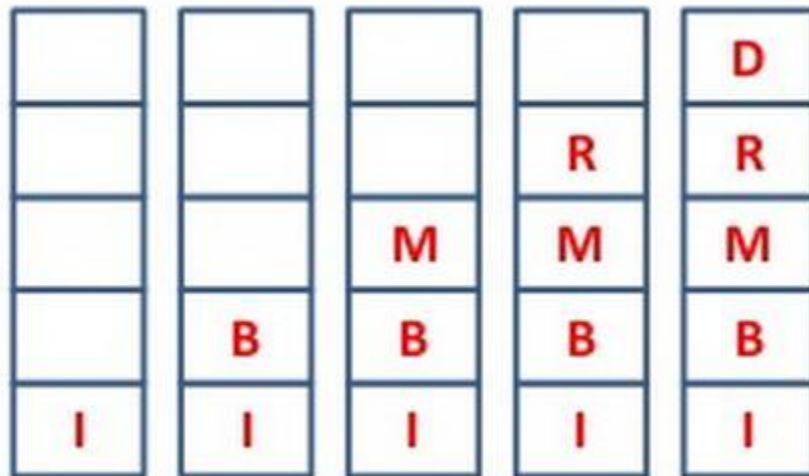
Adds a new element to the top of this stack.

Returns the position of the specified element in this stack.

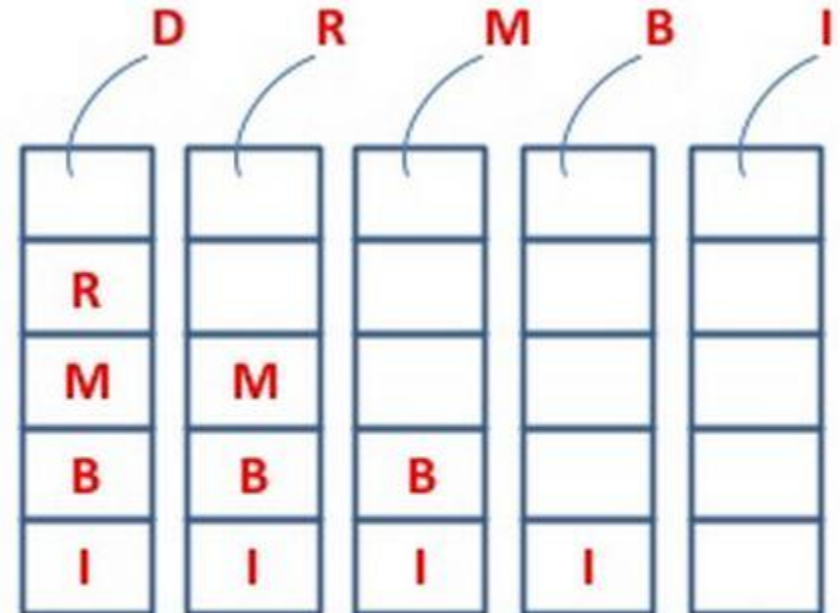
Problem: Reversing a String

E.g.

I/P String is **I B M R D**



PUSH Operation



POP Operation

O/P String is **D R M B I**

Sample Code:

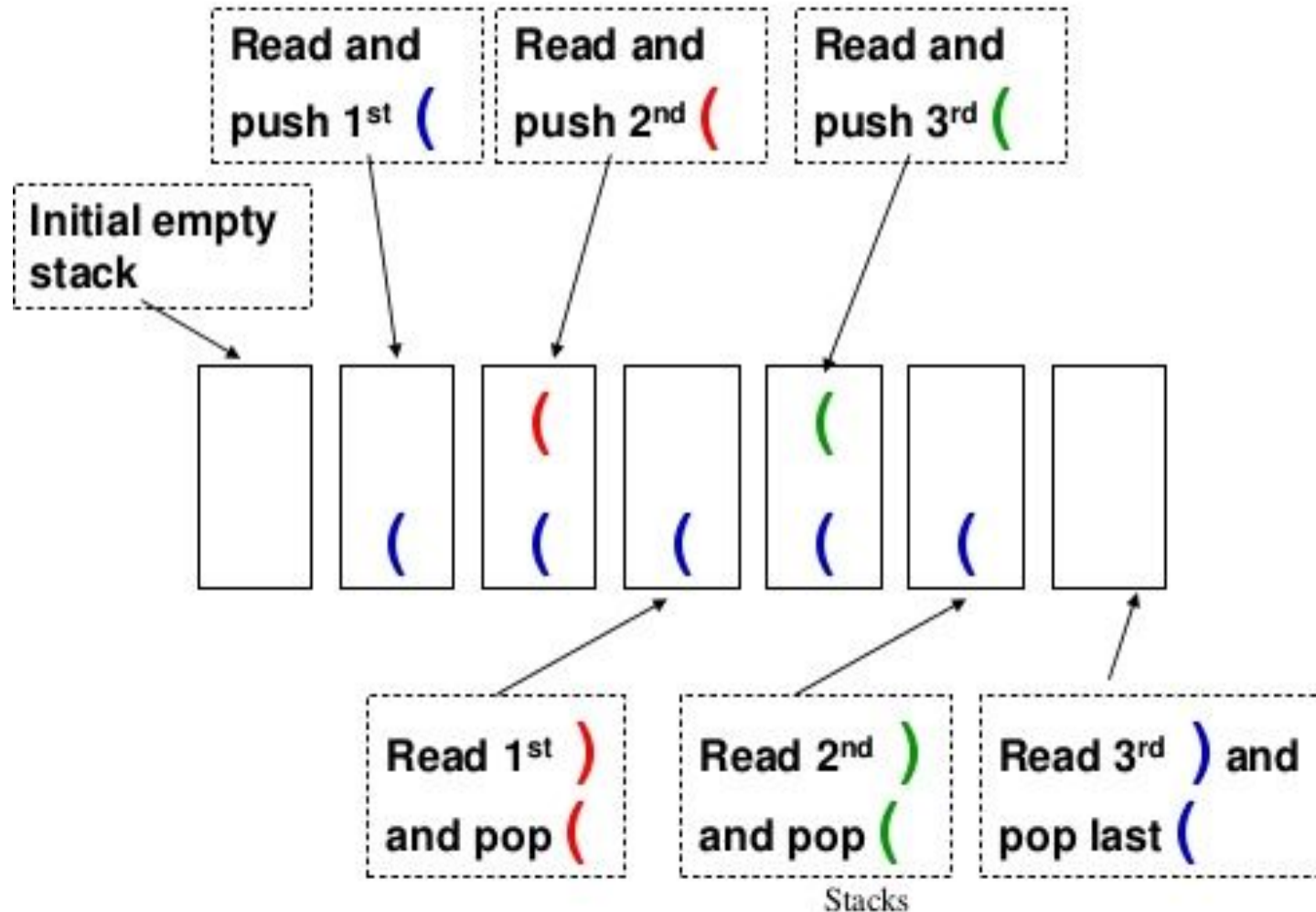
Reversing a String

In **Chapter1\examples\StringToolkit**

- Method **reverse()** in **StringToolkit.java**

Problem: Matching Brackets

Input: (())



Checking for Balanced $()$, $[]$, $\{\}$ (1/7)

- Use a stack to store the **left brackets**
- Brackets include
 - parentheses $()$
 - square brackets $[]$ and
 - braces $\{\}$

Checking for Balanced $()$, $[]$, $\{\}$ (2/7)

- Traverse the expression to process each character
 - If it is a left/open bracket, push it onto the stack.
 - If it is a right/close bracket, pop a bracket from the stack and compare whether the two bracket types match.

Checking for Balanced `()`, `[]`, `{ }` (3/7)

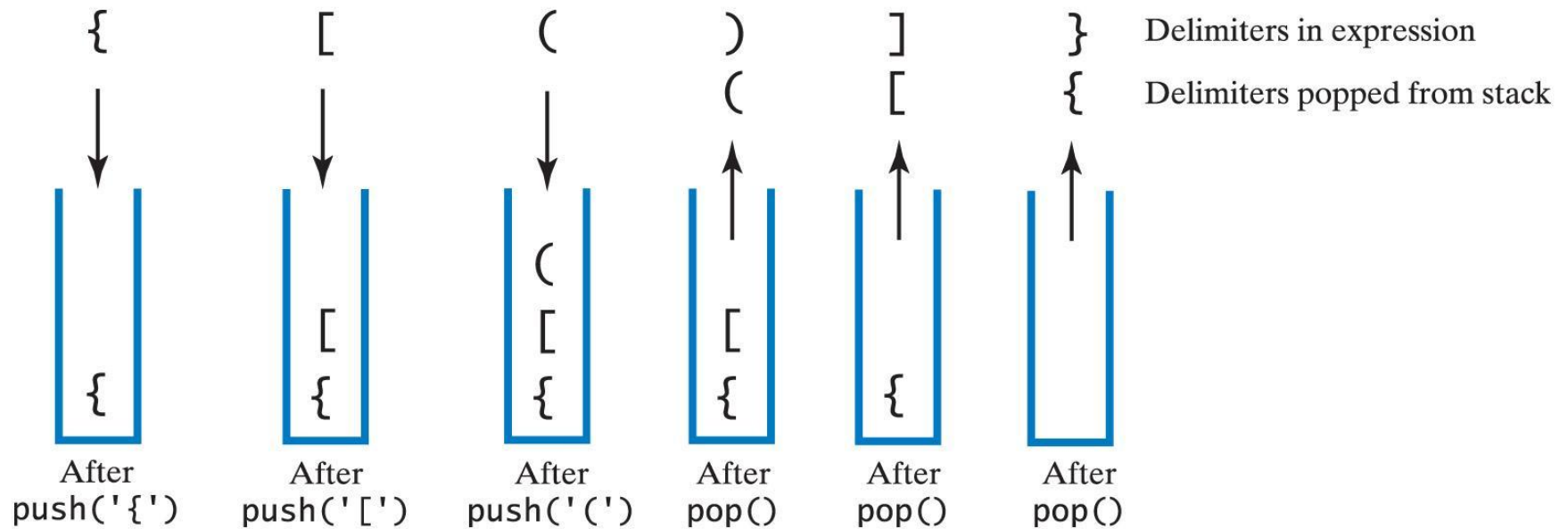


Fig. 21-3 The contents of a stack during the scan of an expression that contains the balanced delimiters `{ [ ( ) ] }`

Checking for Balanced () , [] , { } (4/7)

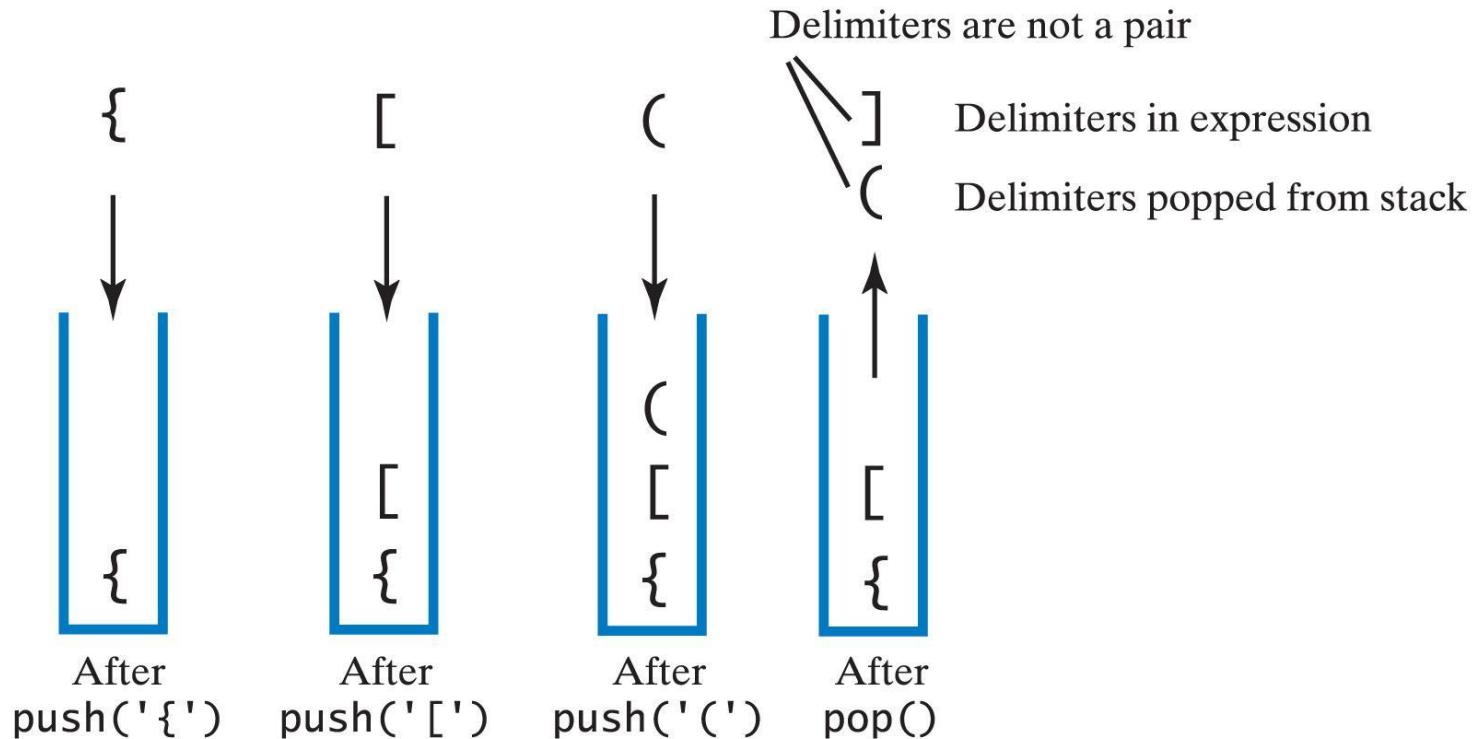


Fig. 21-4 The contents of a stack during the scan of an expression that contains the unbalanced delimiters `{ [ ( ] ) }`

Checking for Balanced () , [] , { } (5/7)

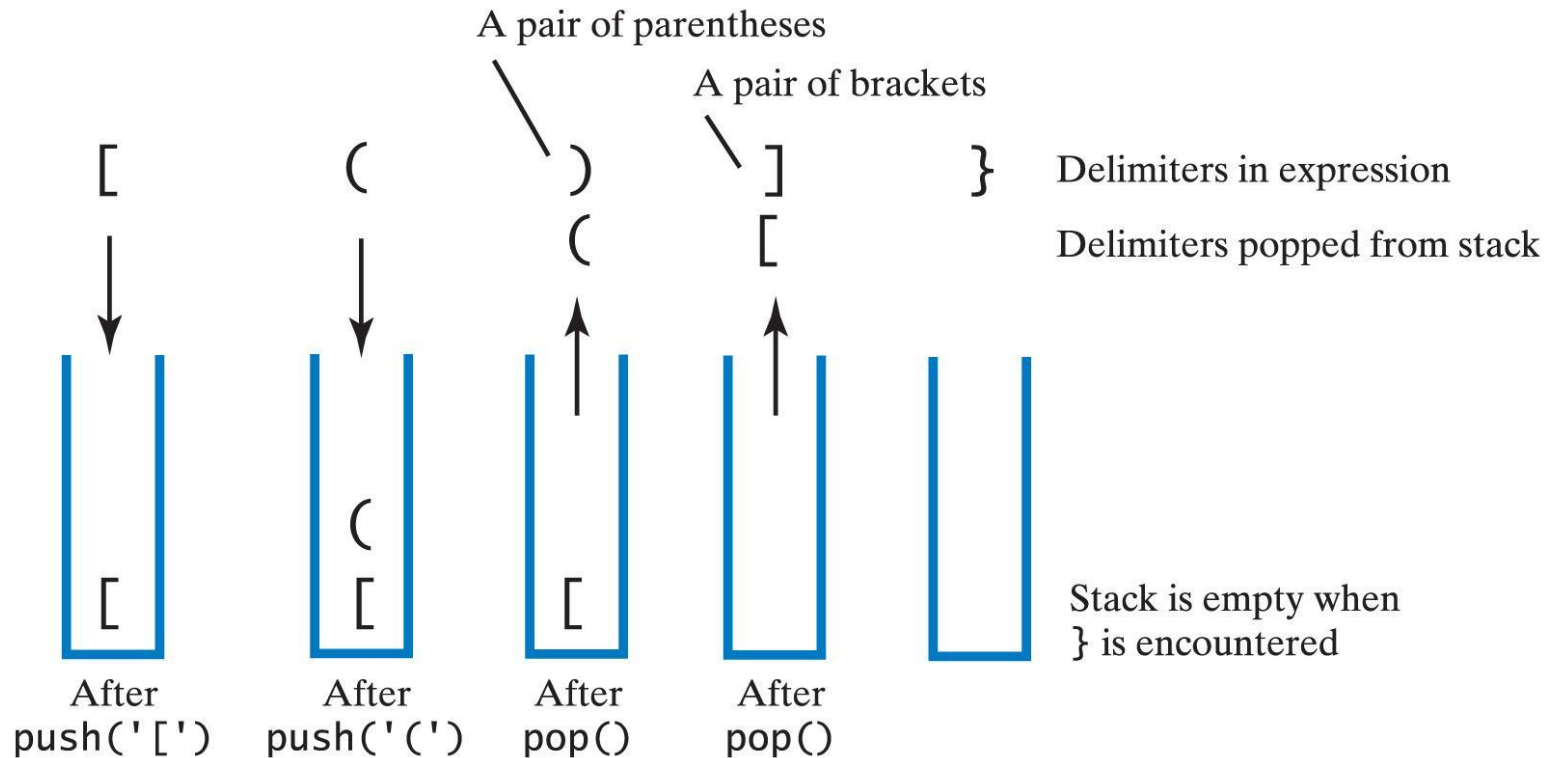


Fig. 21-5 The contents of a stack during the scan of an expression that contains the unbalanced delimiters [()] }

Checking for Balanced () , [] , { } (6/7)

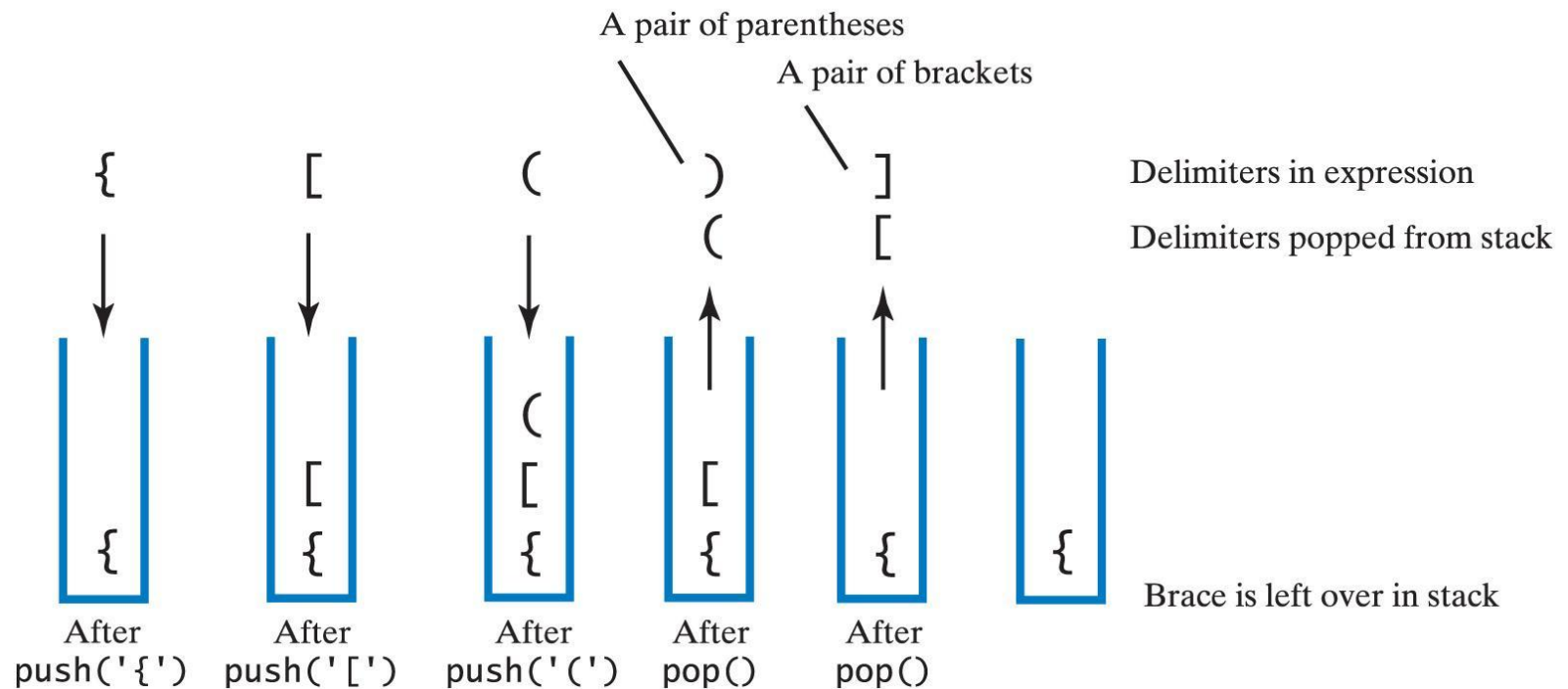


Fig. 21-6 The contents of a stack during the scan of an expression that contains the unbalanced delimiters `{ [ ( ) ]`

Checking for Balanced $()$, $[]$, $\{\}$ (7/7)

- Algorithm 1.1 **checkBalance** - for checking for balanced parentheses in a string

Infix, Prefix and Postfix Expressions

- Infix expressions
 - Binary operators appear between operands
 - **a + b**
- Prefix expressions
 - Binary operators appear before operands
 - **+ a b**
- Postfix expressions
 - Binary operators appear after operands
 - **a b +**
 - Easier to process – no need for parentheses nor precedence



Exercise 1.1

Transform each of the following infix expressions to its postfix and prefix forms:

<u>Infix</u>	<u>Postfix</u>	<u>Prefix</u>
(a) $a + b * c$		
(b) $a * b / (c - d)$		
(c) $a / b + (c - d)$		
(d) $a / b + c - d$		

Problem:
Evaluating
Postfix
Expressions

4	5	+	3	*	7	-
	=	9	3	*		
			=	27	7	-
					=	20

Evaluating Postfix Expression (1 / 3)

- Use a *stack of operands*
- Traverse the expression to process each character
 - If it is an operand, push it onto the stack
 - If it is an operator,
 - Pop the top two elements from the stack,
(Note: the first pop execution returns the right operand while the second pop returns the left operand)
 - Perform the operation on the two elements, and
 - Push the result of the operation onto the stack

Evaluating Postfix Expression (2/3)

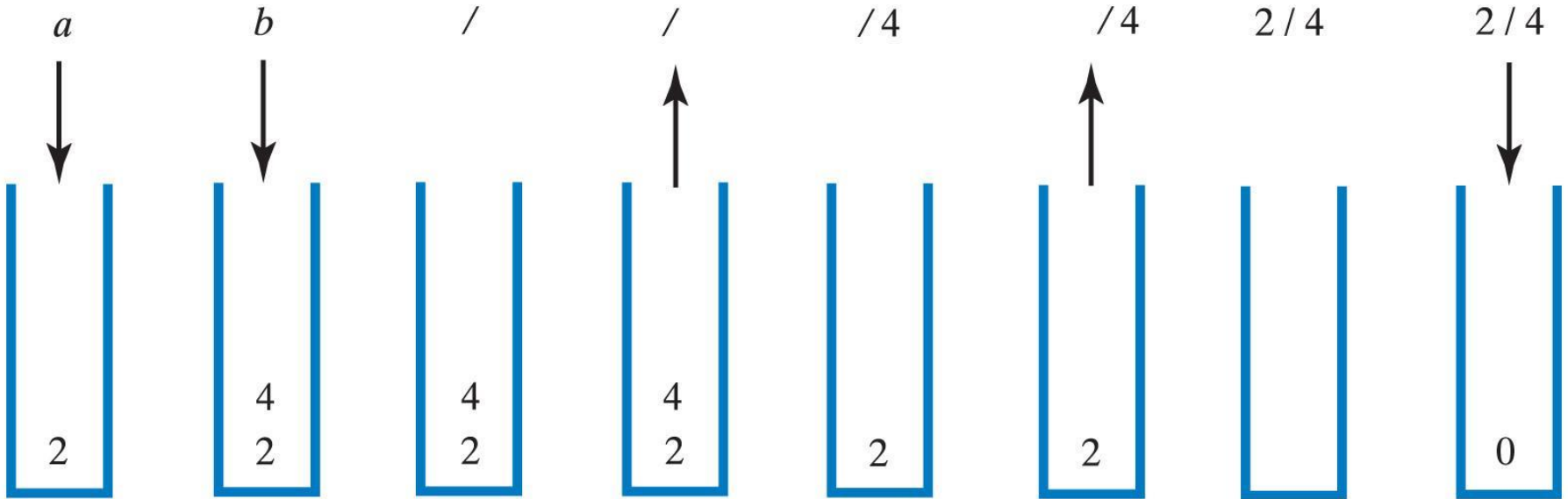


Fig. 21-10 The stack during the evaluation of the postfix expression **a b /** when **a** is 2 and **b** is 4

Evaluating Postfix Expression (3/3)

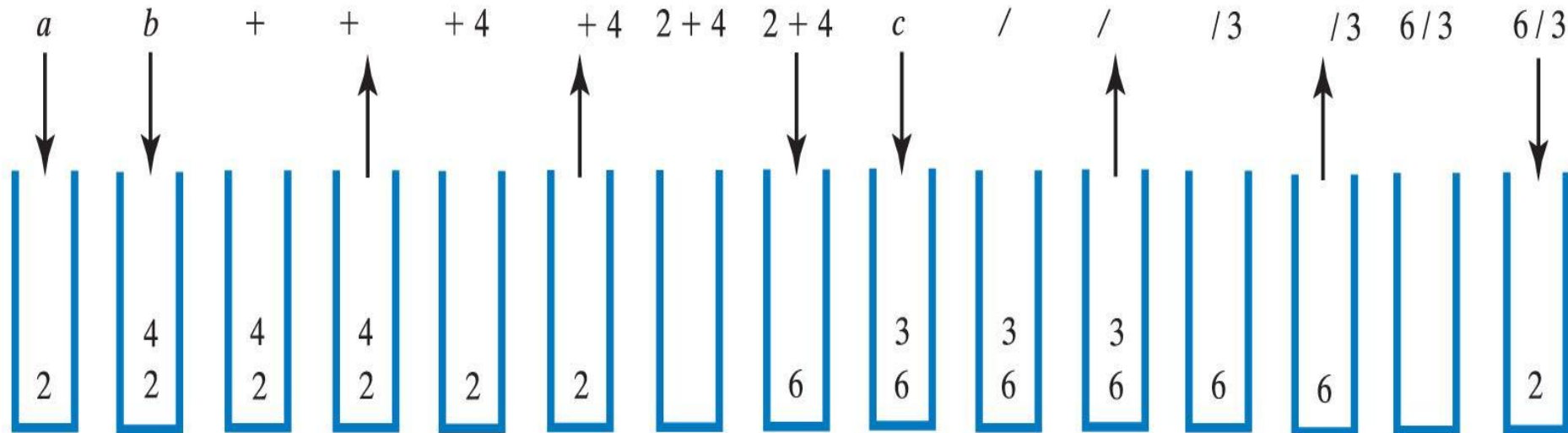


Fig. 21-11 The stack during the evaluation of the postfix expression $a \ b \ + \ c \ /$ when a is 2, b is 4 and c is 3

The Program Stack (1 / 3)

- There is an area of memory in your computer known as the system/program stack which is used to allocate memory for method calls.
- An *activation record* is
 - A portion of memory on the program stack allocated to a method call
 - Used to store the called method's arguments, local variables and reference to the current instruction (program counter)

The Program Stack (2/3)

- When a method is called
 - Runtime environment creates activation record
 - Shows method's state during execution
- Activation record pushed onto the program stack
 - Top of stack belongs to currently executing method
 - Next method down is the one that called current method

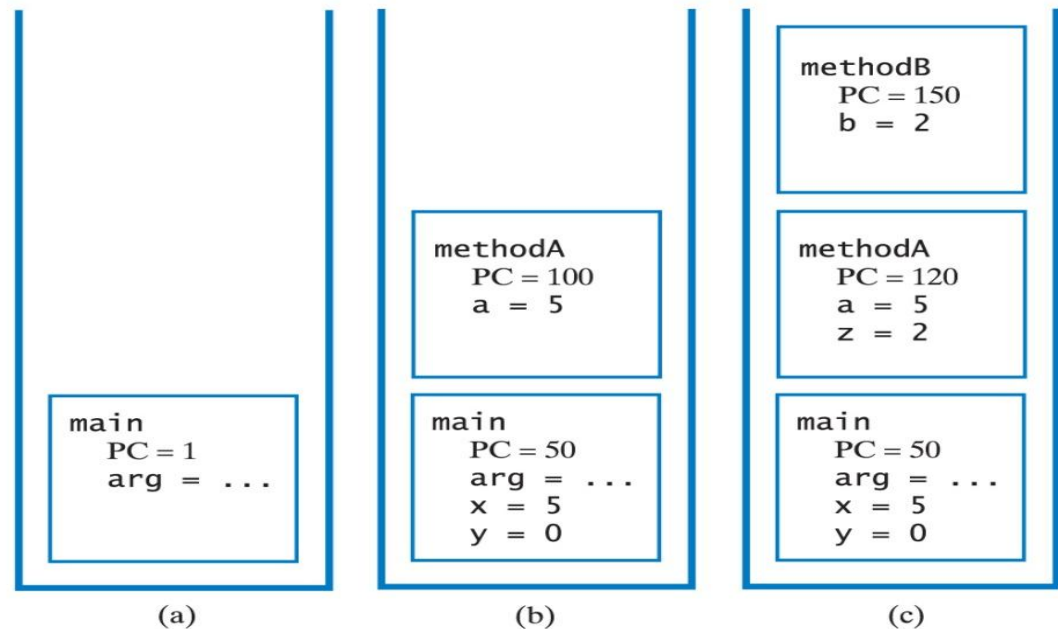
The Program Stack (3/3)



```
1  public static
void main(string[] arg)
{
    int x = 5;
50  int y = methodA(x);
} // end main

100 public static
int methodA(int a)
{
    int z = 2;
120  methodB(z);
    return z;
} // end methodA

150 public static
void methodB(int b)
{
} // end methodB
```



Program

Program stack at three points in time (PC is the program counter)

Fig. 21-13 The program stack at 3 points in time; (a) when **main** begins execution; (b) when **methodA** begins execution, (c) when **methodB** begins execution.

Queues

Queue: Definition

- A collection of objects that are inserted and removed according to the first-in, first out (**FIFO**) principle. That is, the element that has been in the queue the longest will be the next one to be removed. Elements enter a queue at the **rear** or **back**; elements are removed from the **front**
- A linear collection of entries in which entries may be only removed in the order in which they are added. Typically, there is not provision to search for an entry in the queue.

Queue: Basic Operations

Operation	Description
enqueue	Add an entry to the rear of the queue
dequeue	Remove the entry at the front of the queue
isEmpty	Check if the queue is empty
getFront	Retrieve (but do not remove) the front object in the queue
size	Return the number of entries in the queue

Java Collections Framework:

`java.util.Queue` interface

Modifier and Type	Method and Description
boolean	add(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning true upon success and throwing an <code>IllegalStateException</code> if no space is currently available.
E	element() Retrieves, but does not remove, the head of this queue.
boolean	offer(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
E	peek() Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
E	poll() Retrieves and removes the head of this queue, or returns null if this queue is empty.
E	remove() Retrieves and removes the head of this queue.

Java Collections Framework :

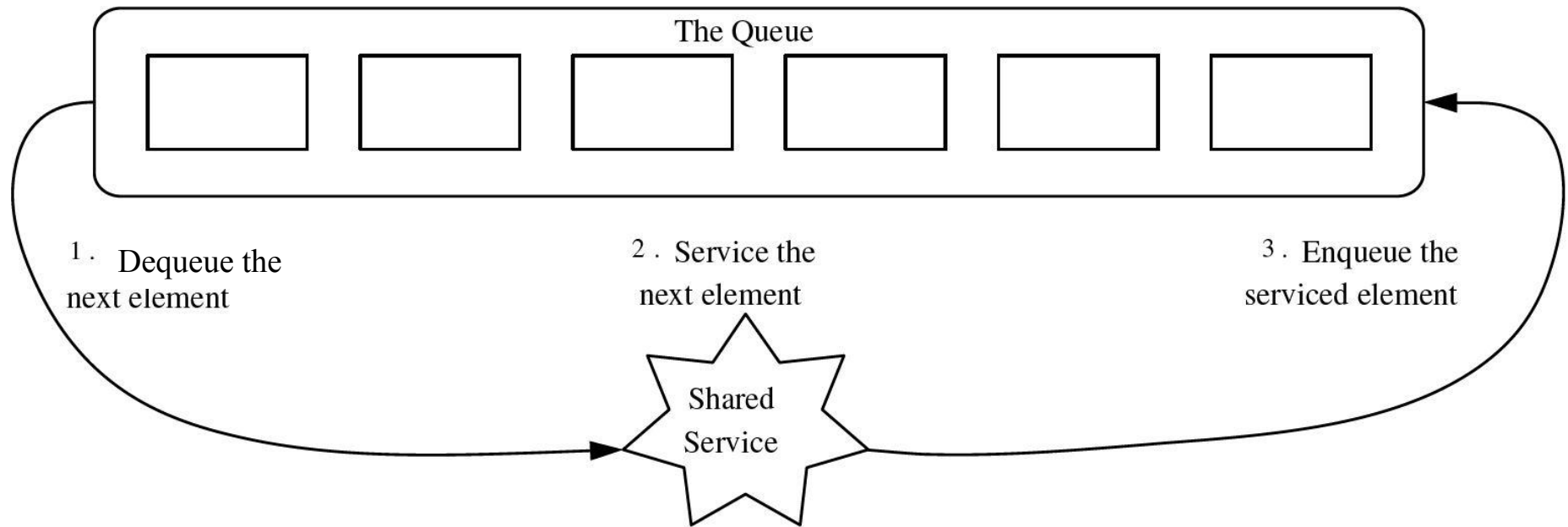
ArrayBlockingQueue class

- **java.util.concurrent.ArrayBlockingQueue** is a concrete class that implements the `java.util.Queue` interface to support the FIFO principle.

Queue: Applications

- Processing of customer requests – *e.g.*, handling calls to the reservation center of a restaurant
- Print queues
- Round-robin schedulers – *e.g.*, to allocate a slice of CPU time to various applications running concurrently on a computer
- Simulation of queues

Problem: Round Robin Schedulers



Problem: Service Counters

- Single queue, multi-counters – *e.g.*, people at the Post Office take a number and wait for their number to be called

Problem: Using a Queue to Simulate a Waiting Line

Application:

- Businesses want to analyze the time that their customers must wait for service.
 - Waiting time ☐
 - Customer satisfaction ☐
 - No. of customers served ☐
 - Profits \$\$ ☐ 

Computer Simulation

- Computer simulation of a real-world situation is a common way to test various business scenarios.
- *E.g.* to simulate a waiting line with one serving agent (i.e. a line of people waiting for service at a single counter)
 - Customers arrive at different intervals and require various times to complete their transactions

Problem:

Computing Profit in Sale of Shares

- Typically, people buy shares in a particular company over a period of time. E.g.,
 - Last year you bought 20 shares of Presto Pizza at RM45 per share;
 - Last month, you bought 20 additional shares at RM75 per share; and
 - Today you sold 30 shares at RM65 per share.

What is your profit? Which of your 40 shares did you sell?

- When computing profit in the sale of shares, you must assume that you sell shares in the order in which you purchased them (i.e. FIFO).
- Design a way to record your investment transactions chronologically and to compute the profit of any sale of shares.

Queue of Shares of Stock

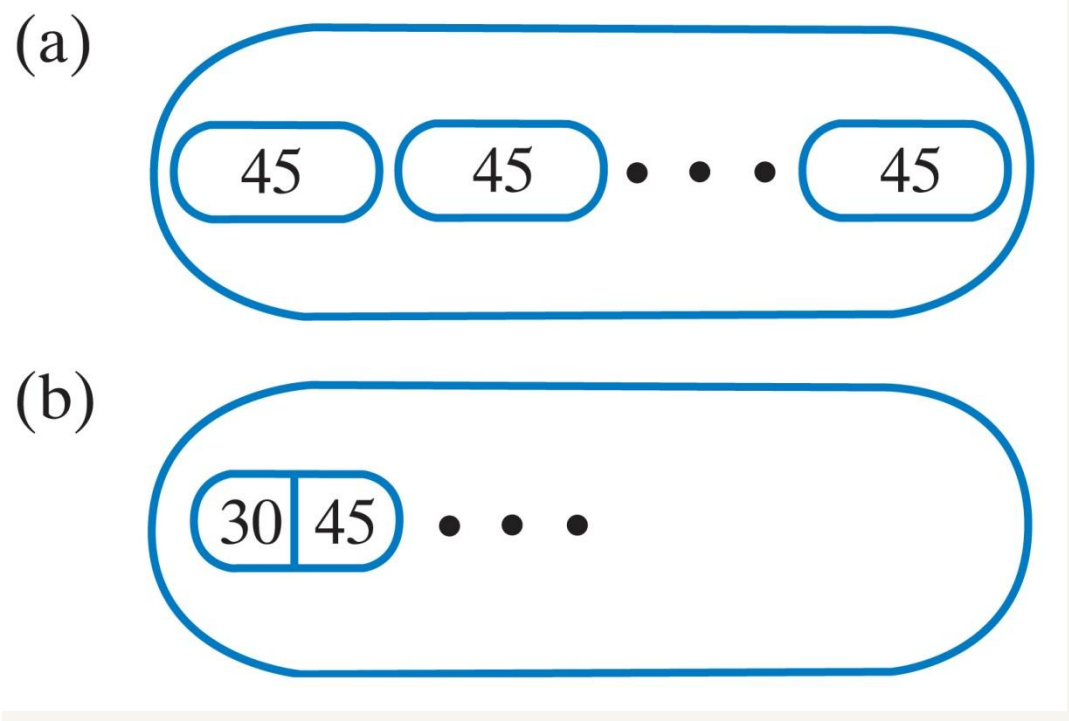


Fig. 23-9 A queue of (a) individual shares of stock;
(b) grouped shares.

Sample Code:

Computing Profit in Sale of Shares

In **Chapter1\examples**

a) **SharePurchase.java**

- A **SharePurchase** object stores the cost of a single share.

b) **SharePortfolio.java**

- A **SharePortfolio** object records stock purchases in chronological order using a queue.
- Note constructor and methods **buy** and **sell**

c) **SharePortfolioDriver.java**



Exercise 1.2

For each application below, identify if a stack or a queue would be the appropriate data structure. Provide justifications for your choices.

- (a) page-visited history in a web browser
- (b) access to shared resources
- (c) undo-sequence in a text editor

Learning Outcomes

You should be able to

- Explain the benefits of *abstraction*.
- Apply the use of the *list*, *stack* and *queue*.
ADTs appropriately to solve problems.